

Diseño de Software de Dispositivos Móviles

Unidad 6

Diseño de Aplicaciones Móviles

Interfaces interactivas, navegación, estado, eventos, integración con APIs y validación funcional

Asignatura: Diseño de Software de Dispositivos Móviles

Carrera: Tecnicatura Universitaria en Diseño de Software - FTyCA, UNCA

Docente: Mg. Ing. Carlos Acosta Parra

Actividad asociada: Desarrollo de una versión funcional con navegación y consumo de una API pública

Entregable: Código fuente versionado en GitHub + demo funcional grabada de hasta 5 minutos

Esta unidad acompaña el pasaje del prototipo validado a una primera aplicación funcional, acotada pero real, capaz de navegar, responder a eventos, gestionar estado y consumir datos externos.

Objetivos de aprendizaje

Al finalizar esta unidad, el estudiante será capaz de:

1. Transformar un prototipo móvil validado en una primera versión funcional de aplicación.
2. Diseñar estructuras de pantallas y flujos de navegación coherentes con las tareas del usuario.
3. Implementar interfaces interactivas que respondan correctamente a eventos, entradas y cambios de estado.
4. Comprender la gestión de estado como mecanismo central para mantener sincronizada la interfaz con los datos.
5. Integrar una aplicación móvil con APIs y servicios externos, contemplando carga, error, latencia y datos vacíos.
6. Aplicar buenas prácticas de prueba y validación funcional antes de presentar una demo.
7. Organizar el código fuente y el repositorio de manera comprensible para el trabajo colaborativo y la evaluación.

Introducción

En la unidad 5 se trabajó sobre el prototipo: una representación navegable, evaluable y todavía económica de modificar. La unidad 6 da un paso decisivo: pasar del prototipo a una aplicación funcional. Este cambio no significa simplemente "programar las pantallas", sino convertir decisiones de diseño en comportamiento real.

Una aplicación móvil funcional debe responder a acciones del usuario, navegar entre pantallas, mantener información en memoria, actualizar la interfaz cuando cambian los datos, consumir servicios externos, manejar errores y ofrecer una experiencia comprensible incluso cuando las condiciones no son ideales. En un dispositivo móvil esas condiciones cambian todo el tiempo: la conexión puede fallar, la pantalla puede rotar, el sistema puede suspender la app, el usuario puede recibir una llamada, volver más tarde o tocar dos veces el mismo botón.

Por eso, el diseño de aplicaciones móviles no termina en la apariencia visual. También incluye el diseño de flujos, estados, eventos, reglas de interacción, manejo de datos, tiempos de respuesta y validaciones. Una pantalla bonita pero incapaz de responder bien a un error de red no ofrece una buena experiencia. Del mismo modo, una app técnicamente correcta pero con navegación confusa tampoco resuelve el problema del usuario.

La meta de esta unidad es construir una primera versión funcional, acotada pero real: una aplicación que tenga varias pantallas conectadas, interacciones significativas y consumo de una API pública. No se espera todavía un producto final listo para tienda, pero sí una base sólida para continuar con integración, autenticación, almacenamiento y publicación en las unidades siguientes.

Idea central

Una aplicación móvil funcional no es un conjunto de pantallas aisladas: es un sistema interactivo donde navegación, estado, eventos, datos y validación trabajan juntos para sostener la experiencia del usuario.

1. Del prototipo a la aplicación funcional

El prototipo permitió responder preguntas de diseño: si el flujo se entiende, si la acción principal es visible, si la interfaz respeta criterios de jerarquía, accesibilidad y consistencia. La aplicación funcional debe responder una pregunta distinta: si ese diseño puede ejecutarse con datos, lógica e interacción real.

En esta transición aparecen nuevas decisiones:

- Qué pantallas se implementan primero.
- Qué componentes se reutilizan.
- Qué datos serán locales y cuáles vendrán de servicios externos.
- Qué estados necesita representar la interfaz.
- Qué rutas de navegación existen y qué parámetros se pasan entre pantallas.
- Qué errores pueden ocurrir y cómo se comunican.
- Qué parte del comportamiento será simulada y qué parte será real.

El objetivo no es implementar "todo", sino construir una primera versión útil, demostrable y coherente. Para lograrlo conviene trabajar con el concepto de **vertical slice**: una porción vertical del producto que atraviesa interfaz, navegación, lógica y datos de extremo a extremo.

Del prototipo a la aplicación funcional

Una primera versión completa atraviesa diseño, código, datos y validación



Figura 1. Del prototipo validado a una primera aplicación funcional.

1.1 Qué significa una versión funcional

Una versión funcional es una app que permite realizar una tarea concreta con comportamiento real, aunque el alcance sea limitado. Por ejemplo:

- Buscar información en una API pública y ver un detalle.
- Listar elementos, filtrarlos y navegar a una pantalla de información ampliada.
- Completar un formulario, validar campos y mostrar un resultado.
- Consultar datos remotos, mostrar estados de carga y manejar errores.
- Guardar una preferencia simple durante la sesión.

No alcanza con que las pantallas existan. La aplicación debe responder al usuario.

Aspecto	Prototipo	Versión funcional
Pantallas	Simuladas o estáticas	Implementadas en código
Navegación	Hotspots o enlaces ficticios	Rutas reales entre vistas

Aspecto	Prototipo	Versión funcional
Datos	Ejemplos escritos a mano	Datos locales o remotos
Interacción	Simulada	Eventos capturados y procesados
Estado	Representado visualmente	Gestionado por la app
Errores	Muchas veces ausentes	Deben mostrarse y recuperarse
Validación	Principalmente con usuarios	Con usuarios y pruebas técnicas

1.2 Alcance recomendado

Para el trabajo práctico de la unidad, el alcance debe ser pequeño pero completo. Una buena primera versión suele incluir:

- Pantalla de inicio o listado principal.
- Pantalla de detalle.
- Navegación entre al menos dos pantallas.
- Consumo de una API pública.
- Indicador de carga.
- Manejo de error de red o respuesta inválida.
- Estado vacío cuando no hay datos.
- Interacción del usuario: búsqueda, filtro, selección, formulario breve o actualización.
- Diseño visual consistente con el prototipo de la unidad anterior.

Este alcance permite evaluar la cadena completa de diseño y desarrollo sin convertir la actividad en un proyecto demasiado grande.

1.3 Riesgos frecuentes al pasar a código

Al implementar, suelen aparecer problemas que el prototipo no mostraba:

- El contenido real es más largo que el texto de ejemplo y rompe el layout.
- La API devuelve campos nulos, listas vacías o estructuras inesperadas.
- La navegación permite volver a pantallas en estados inconsistentes.
- El botón principal se puede presionar muchas veces y genera acciones duplicadas.
- Los errores técnicos se muestran como mensajes incomprensibles.
- La app depende de conexión permanente y no comunica qué ocurre si falla.
- El diseño no contempla teclado en pantalla, áreas seguras o tamaños de fuente del sistema.

Estos riesgos no son fallas raras. Son parte normal del desarrollo móvil. Diseñar una aplicación implica anticiparlos y construir una interfaz capaz de seguir siendo clara cuando el camino feliz se interrumpe.

2. Estructura de pantallas y arquitectura de interfaz

Una app móvil se organiza como un conjunto de pantallas, componentes y flujos. Cada pantalla tiene una responsabilidad: mostrar información, permitir una decisión, capturar datos, confirmar una acción o comunicar un resultado.

Antes de programar conviene responder tres preguntas:

- ¿Cuál es la tarea principal que esta pantalla permite realizar?
- ¿Qué información necesita el usuario para avanzar?
- ¿Qué acciones son posibles y cuál es la prioritaria?

Si una pantalla intenta resolver demasiadas tareas a la vez, la interfaz se vuelve confusa y el código también. La claridad visual y la claridad técnica suelen ir juntas.

2.1 Pantallas, vistas y componentes

Aunque cada framework utiliza nombres distintos, la lógica general es similar:

- **Pantalla:** unidad completa de navegación. Ejemplo: Inicio, Detalle, Perfil, Configuración.
- **Vista o sección:** parte interna de una pantalla. Ejemplo: encabezado, lista de resultados, formulario.
- **Componente:** elemento reutilizable. Ejemplo: botón, tarjeta, campo de texto, barra de búsqueda.
- **Layout:** estructura que ordena los elementos. Ejemplo: columna, fila, grilla, lista desplazable.
- **Estado visual:** variante de una pantalla o componente. Ejemplo: cargando, vacío, error, éxito.

Separar estos niveles ayuda a evitar duplicación y facilita el mantenimiento.

2.2 Componentización

Componentizar significa dividir la interfaz en partes reutilizables y comprensibles. No se trata de crear componentes para todo, sino de identificar patrones que se repiten o que tienen una responsabilidad clara.

Ejemplo: en una app que lista recetas, una tarjeta de receta puede incluir imagen, nombre, categoría, tiempo estimado y acción para ver detalle. Esa tarjeta debería ser un componente reutilizable, porque puede aparecer en la pantalla principal, en favoritos o en resultados de búsqueda.

Un buen componente:

- Tiene un propósito claro.
- Recibe datos desde afuera.
- Evita depender de variables globales innecesarias.
- Expone eventos comprensibles, como `onPress`, `onSelect` o `onSubmit`.
- Maneja sus variantes visuales sin duplicar código.
- Respeta accesibilidad, tamaño táctil y consistencia visual.

2.3 Separación entre interfaz, lógica y datos

En una aplicación pequeña es tentador mezclar todo dentro de una misma pantalla: código visual, llamada a la API, transformación de datos, validaciones y navegación. Esto funciona al principio, pero rápidamente vuelve difícil modificar o probar la app.

Una separación simple puede organizarse así:

Arquitectura simple de una app móvil

Separar responsabilidades ayuda a leer, probar y modificar el proyecto

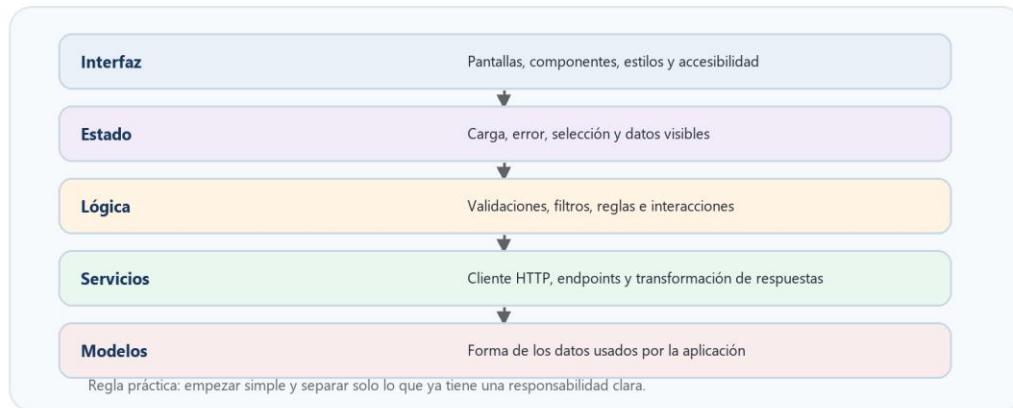


Figura 2. Capas mínimas para organizar una aplicación móvil funcional.

Capa	Responsabilidad	Ejemplos
Interfaz	Mostrar datos y capturar acciones del usuario	Pantallas, componentes, estilos
Estado	Recordar qué está ocurriendo en la pantalla	Cargando, lista de datos, error, selección
Lógica	Tomar decisiones y transformar información	Validaciones, filtros, ordenamientos
Servicios	Comunicarse con APIs o recursos externos	Cliente HTTP, endpoints, parseo de respuestas
Modelos	Definir la forma de los datos	Usuario, producto, receta, turno

No hace falta aplicar una arquitectura compleja en el primer proyecto. Sí es importante evitar que toda la aplicación quede concentrada en un único archivo difícil de leer.

2.4 Diseño para contenido real

El prototipo suele usar textos controlados. La aplicación funcional recibe datos reales o semirreales, por lo que debe prepararse para variaciones:

- Títulos largos.
- Imágenes ausentes.
- Descripciones extensas.
- Fechas en formatos diferentes.
- Datos incompletos.
- Listas con cero, pocos o muchos elementos.

- Caracteres especiales o idiomas distintos.

Una buena pantalla móvil no depende de que todos los datos sean perfectos. Debe degradar con elegancia: truncar cuando corresponde, permitir desplazamiento, mostrar valores alternativos y evitar que la interfaz se rompa.

3. Flujos de navegación

La navegación define cómo se mueve el usuario dentro de la aplicación. En móvil, navegar no es solo cambiar de pantalla: es sostener una sensación de ubicación, continuidad y control.

Una app con buena navegación responde siempre estas preguntas:

- ¿Dónde estoy?
- ¿De dónde vengo?
- ¿Qué puedo hacer ahora?
- ¿Cómo vuelvo?
- ¿Qué ocurrió después de mi acción?

3.1 Patrones principales de navegación móvil

Los patrones más comunes son:

Patrones de navegación móvil

Elegir el patrón según la tarea evita flujos confusos

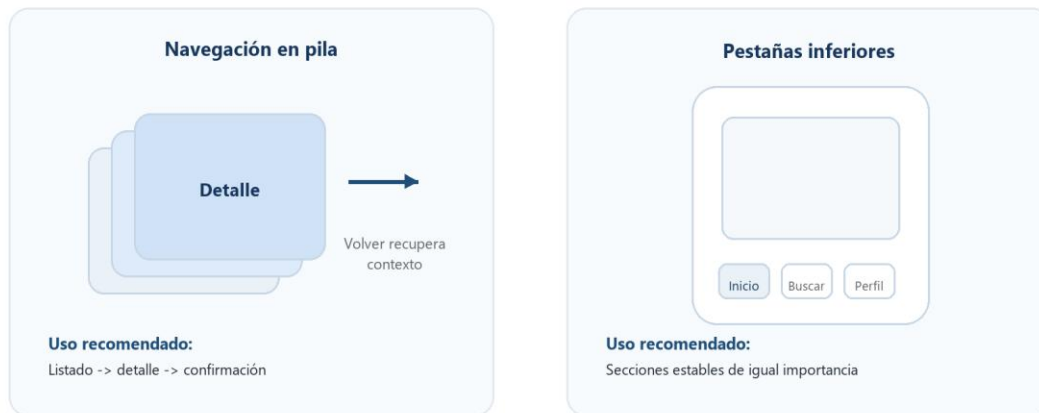


Figura 3. Dos patrones de navegación habituales en aplicaciones móviles.

La siguiente galería resume visualmente los patrones principales de navegación móvil y cuándo conviene utilizarlos.

Galería de patrones de navegación estructural

Patrones que organizan el movimiento principal dentro de la app



Figura 4. Patrones estructurales de navegación: pila, pestañas inferiores y menú lateral.

Galería de patrones contextuales

Patrones para tareas breves, acciones focalizadas o recorridos iniciales

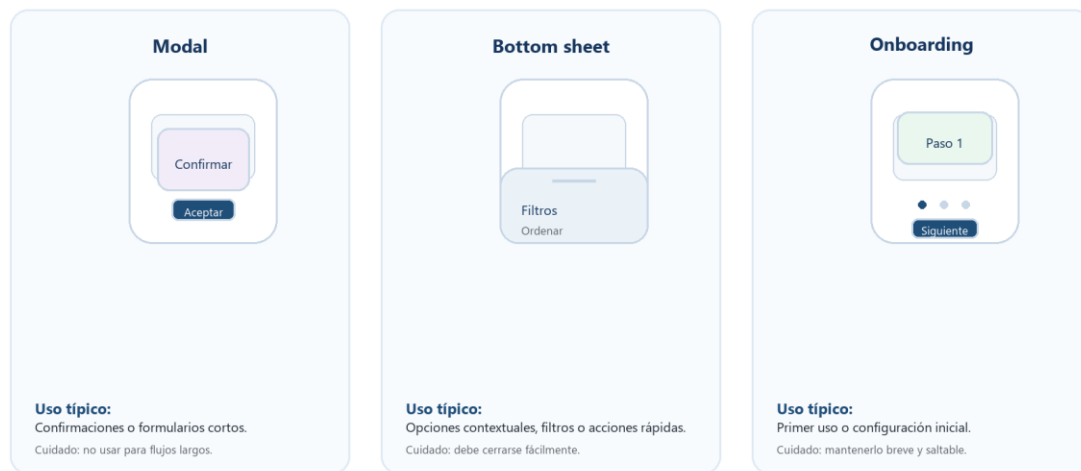


Figura 5. Patrones contextuales de navegación: modal, bottom sheet y onboarding.

Patrón	Uso adecuado	Cuidado
Navegación en pila	Flujos jerárquicos: listado -> detalle -> edición	Evitar pilas demasiado profundas
Pestañas inferiores	Secciones principales de una app	No usar más de 3 a 5 secciones
Menú lateral	Apps con muchas áreas	En móvil puede ocultar opciones

Patrón	Uso adecuado	Cuidado
	secundarias	importantes
Modal	Tarea breve, confirmación o formulario puntual	No usar para flujos largos
Bottom sheet	Opciones contextuales, filtros, detalles rápidos	Debe poder cerrarse fácilmente
Onboarding	Presentación inicial o configuración	Debe ser breve y saltable cuando sea posible

En una primera versión funcional, una navegación en pila suele ser suficiente: pantalla principal, pantalla de detalle y, si corresponde, una pantalla de búsqueda o configuración.

3.2 Navegación en pila

La navegación en pila funciona como una pila de tarjetas: cada nueva pantalla se coloca encima de la anterior. Al volver, la pantalla superior se retira y se recupera la anterior.

Ejemplo:

```

Inicio
  -> Lista de lugares
    -> Detalle del lugar
      -> Mapa o información ampliada
  
```

Este patrón es natural para relaciones jerárquicas. El usuario entiende que puede entrar en un detalle y luego regresar.

Buenas prácticas:

- Mantener títulos claros en cada pantalla.
- Usar el botón o gesto de volver según la plataforma.
- Evitar que una acción de confirmación deje al usuario atrapado.
- No duplicar la misma pantalla muchas veces con estados difíciles de distinguir.
- Pasar a la pantalla siguiente solo los datos necesarios.

3.3 Pestañas inferiores

Las pestañas inferiores permiten cambiar entre secciones principales de la app. Son útiles cuando existen áreas de igual importancia, por ejemplo Inicio, Buscar, Favoritos y Perfil.

Una pestaña no debería representar un paso dentro de un flujo, sino una zona estable de la aplicación. Si el usuario está completando un formulario o siguiendo una secuencia, lo más probable es que convenga una navegación en pila, no pestañas.

Recomendaciones:

- Usar etiquetas breves y comprensibles.
- Acompañar con íconos claros cuando sea posible.

- Mantener siempre el mismo orden.
- No esconder acciones críticas dentro de pestañas secundarias.
- Evitar cambiar el significado de una pestaña según el contexto.

3.4 Parámetros de navegación

Muchas pantallas necesitan recibir información desde la pantalla anterior. Por ejemplo, al tocar una receta en una lista, la pantalla de detalle necesita saber qué receta mostrar.

Existen dos estrategias:

- Pasar el objeto completo.
- Pasar solo un identificador y volver a cargar el detalle.

Pasar el objeto completo es simple y rápido para apps pequeñas. Pasar el identificador es más robusto cuando el detalle debe actualizarse desde una API, cuando el objeto es grande o cuando puede abrirse la pantalla desde una notificación o enlace profundo.

Ejemplo conceptual:

```

Lista de productos
  usuario toca producto con id = 42
Detalle de producto
  recibe id = 42
  consulta datos del producto
  muestra carga, éxito o error
  
```

3.5 Navegación y modelo mental

El flujo técnico debe coincidir con el modelo mental del usuario. Si la persona piensa "primero elijo una ciudad y después veo el clima", la app debería respetar esa secuencia. Si la app obliga a elegir una ciudad, crear una cuenta, aceptar permisos y recién después mostrar información, puede generar abandono.

Una forma simple de revisar navegación es escribir el flujo como una historia:

```

Como usuario quiero buscar una ciudad, ver el clima actual y guardar mis favoritas para
consultarlas rápido más tarde.
  
```

Luego se verifica si las pantallas acompañan esa historia sin desvíos innecesarios.

4. Interfaces interactivas

Una interfaz interactiva responde al usuario. Cada toque, gesto, escritura, selección o desplazamiento puede disparar un evento. El diseño debe definir qué ocurre antes, durante y después de esa acción.

En móvil, la interacción tiene características particulares:

- Se realiza con dedos, no con un puntero preciso.
- Puede ocurrir con una sola mano.
- Se desarrolla en sesiones breves e interrumpibles.
- Depende de sensores, teclado virtual, orientación, permisos y conectividad.
- Requiere feedback rápido para evitar incertidumbre.

4.1 Eventos de usuario

Un evento es algo que ocurre y que la aplicación puede detectar. Los más comunes son:

- Tocar un botón.
- Seleccionar un elemento de una lista.
- Escribir en un campo.
- Enviar un formulario.
- Desplazar una lista.
- Refrescar contenido.
- Cambiar una opción.
- Abrir o cerrar un modal.
- Aceptar o denegar un permiso.

Cada evento debería tener una respuesta clara. Si el usuario toca "Buscar", la app debe mostrar que la búsqueda comenzó. Si no hay resultados, debe explicar que no encontró datos. Si hubo error de red, debe comunicarlo con una opción de reintento.

4.2 Feedback inmediato

La retroalimentación evita incertidumbre. En una app funcional, no alcanza con que la acción "funcione internamente"; el usuario debe percibirlo.

Tipos de feedback:

- **Visual:** cambio de color, indicador de carga, mensaje, lista actualizada.
- **Háptico:** vibración breve en acciones importantes, cuando el framework lo permite.
- **Sonoro:** menos frecuente, útil en contextos específicos.
- **De navegación:** cambio de pantalla, cierre de modal, retorno al inicio.
- **De contenido:** aparición de datos, confirmación o comprobante.

Ejemplo: si un botón inicia una llamada a una API, el botón puede quedar deshabilitado mientras se procesa la solicitud y mostrar un indicador de carga. Esto evita toques repetidos y comunica que la app está trabajando.

4.3 Diseño de formularios móviles

Los formularios son uno de los puntos donde más se percibe la calidad de una app. Un mal formulario puede hacer abandonar una tarea aunque la idea del producto sea buena.

Buenas prácticas:

- Pedir solo los datos necesarios.
- Usar el tipo de teclado adecuado: numérico, email, teléfono, texto.
- Validar en el momento correcto, no castigar cada tecla con errores prematuros.
- Mostrar mensajes de error cerca del campo afectado.
- Mantener visible la acción principal cuando el teclado aparece.
- Permitir corregir sin perder información.
- Indicar qué campos son obligatorios.
- Usar valores por defecto cuando ayuden.

Situación	Respuesta recomendada
Campo obligatorio vacío	Mensaje claro: "Ingresá tu nombre para continuar"

Situación	Respuesta recomendada
Email inválido	Explicar formato esperado sin tecnicismos
Contraseña débil	Mostrar criterios de seguridad de forma progresiva
Error del servidor	Diferenciar error del usuario y error del sistema
Envío exitoso	Confirmar y mostrar próximo paso

4.4 Microinteracciones

Las microinteracciones son pequeñas respuestas del sistema que hacen que la interfaz se sienta viva y comprensible. No son decoración: orientan al usuario.

Ejemplos:

- Un botón cambia de estado al presionarse.
- Un campo muestra una marca de verificación cuando el dato es válido.
- Una tarjeta se expande para mostrar más información.
- Una lista se refresca al deslizar hacia abajo.
- Un ícono de favorito cambia al seleccionarse.
- Un mensaje aparece brevemente para confirmar una acción.

La regla es simple: la microinteracción debe aclarar lo que ocurre. Si solo agrega movimiento sin aportar comprensión, probablemente sobre.

4.5 Prevención de acciones duplicadas

En aplicaciones reales, el usuario puede tocar dos veces un botón por ansiedad o porque la app demora. Si la acción envía un formulario, crea una reserva o realiza una compra, esto puede generar problemas.

Medidas habituales:

- Deshabilitar el botón mientras se procesa.
- Mostrar estado de carga.
- Hacer que la operación sea idempotente cuando el backend lo permita.
- Confirmar acciones destructivas.
- Evitar que el mismo evento dispare múltiples solicitudes simultáneas.

5. Gestión de estado

El estado es la memoria actual de la aplicación. Define qué datos tiene, qué está esperando, qué errores ocurrieron, qué seleccionó el usuario y cómo debe verse la interfaz en este momento.

Ciclo de estado de una pantalla con API

La interfaz visible debe derivarse del estado actual

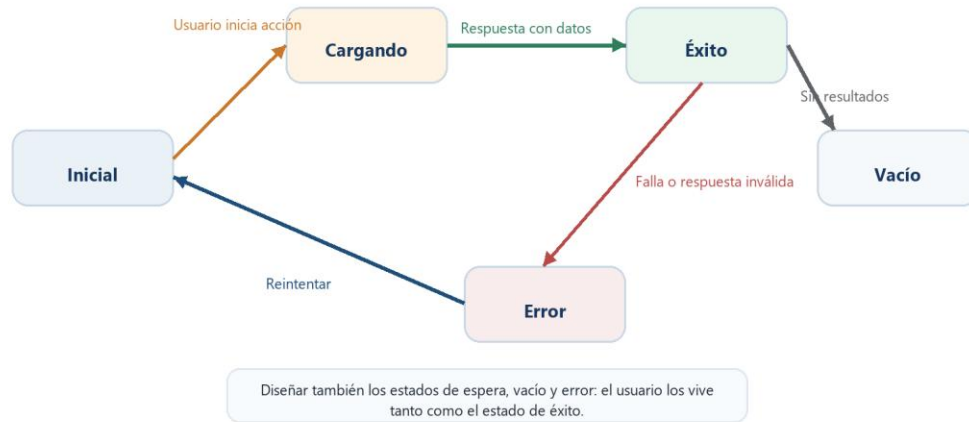


Figura 6. Ciclo de estados típico de una pantalla que consulta datos remotos.

Una interfaz móvil puede pensarse como una función del estado:

estado actual -> interfaz visible

Si el estado cambia, la interfaz debería actualizarse. Este principio es central en frameworks modernos como React Native, Flutter, SwiftUI y Jetpack Compose.

5.1 Estados frecuentes en una pantalla

Una pantalla que consume una API no tiene un solo estado. Como mínimo suele necesitar:

Estado	Qué significa	Qué debería mostrar la UI
Inicial	Todavía no se solicitó información	Vista base o acción para comenzar
Cargando	Hay una solicitud en curso	Indicador de carga o skeleton
Éxito	La solicitud devolvió datos	Contenido principal
Vacío	La solicitud fue correcta pero no hay resultados	Mensaje útil y acción alternativa
Error	La solicitud falló	Explicación breve y botón de reintento
Actualizando	Ya hay datos, pero se están refrescando	Mantener datos visibles y mostrar progreso sutil

Muchos errores de experiencia aparecen porque la app solo diseña el estado de éxito. Pero el usuario también vive los estados de espera, error y vacío.

5.2 Estado local y estado compartido

No todo estado tiene el mismo alcance.

- **Estado local:** pertenece a un componente o pantalla. Ejemplo: texto escrito en un buscador, modal abierto, pestaña seleccionada.
- **Estado compartido:** debe ser leído por varias pantallas. Ejemplo: usuario autenticado, tema visual, lista de favoritos.
- **Estado remoto:** proviene de una API o base de datos. Ejemplo: productos, turnos, clima, noticias.
- **Estado persistente:** se conserva aunque la app se cierre. Ejemplo: preferencias, token de sesión, datos cacheados.

Para una primera versión funcional, lo recomendable es comenzar con estado local y solo elevarlo a un nivel compartido cuando realmente lo necesiten varias pantallas.

5.3 Flujo de datos

Un flujo de datos sano suele seguir esta secuencia:

```

Usuario realiza acción
-> se actualiza estado de carga
-> se solicita información a un servicio
-> se transforma la respuesta
-> se actualiza estado de éxito, vacío o error
-> la interfaz se redibuja
  
```

Este flujo evita que la interfaz quede desordenada. La pantalla no debería "adivinar" qué mostrar; debería derivarlo del estado.

5.4 Ejemplo conceptual de estado

Sin atarse a un framework específico, una pantalla de búsqueda podría manejar algo parecido a esto:

```

estado = {
  consulta: "Catamarca",
  cargando: false,
  datos: [],
  error: null
}
  
```

Al presionar buscar:

```

1. cargando = true
2. error = null
3. llamar API
4. si hay respuesta:
    datos = respuesta
  
```

```

    cargando = false
  si falla:
    error = "No se pudo obtener la información"
    cargando = false

```

Este esquema simple ayuda a razonar antes de escribir código.

5.5 Errores habituales en gestión de estado

- Guardar el mismo dato en muchos lugares distintos.
- Actualizar la interfaz directamente sin actualizar el estado.
- No limpiar errores anteriores al realizar una nueva solicitud.
- Mostrar datos viejos como si fueran nuevos.
- Mezclar estado de formulario con estado de navegación.
- No contemplar qué pasa si la pantalla se cierra mientras una solicitud sigue en curso.
- Usar estado global para información que solo necesita una pantalla.

La buena gestión de estado no busca complicar la app. Busca que el comportamiento sea predecible.

6. Integración de APIs y servicios externos

Una API (Application Programming Interface / Interfaz de Programación de Aplicaciones) permite que una aplicación se comunique con otro sistema. En el contexto de esta unidad, la API pública será la fuente de datos de la app.

Consumir una API no es solo "traer JSON". Implica entender contratos, tiempos de respuesta, errores, transformación de datos, seguridad y experiencia de usuario.

6.1 Qué es una API REST

REST (Representational State Transfer / Transferencia de Estado Representacional) es un estilo de arquitectura para servicios web. En una API REST, los recursos se exponen mediante URLs y se manipulan usando métodos HTTP.

Método	Uso habitual	Ejemplo conceptual
GET	Obtener datos	Obtener lista de productos
POST	Crear un recurso	Crear una reserva
PUT/PATCH	Actualizar un recurso	Modificar perfil
DELETE	Eliminar un recurso	Borrar favorito

Para la actividad de esta unidad, normalmente alcanza con usar [GET](#) sobre una API pública.

6.2 JSON como formato de intercambio

JSON (JavaScript Object Notation / Notación de Objetos JavaScript) es el formato más común para intercambiar datos en aplicaciones móviles y web.

Ejemplo:

```
{
  "id": 42,
  "nombre": "San Fernando del Valle de Catamarca",
  "temperatura": 28,
  "estado": "Soleado"
}
```

La app debe transformar esa respuesta en datos útiles para la interfaz. No siempre conviene mostrar la respuesta tal como llega. A veces hay que renombrar campos, combinar valores, aplicar formato a fechas o descartar información innecesaria.

6.3 Cliente HTTP

Cada plataforma ofrece formas de realizar solicitudes HTTP:

- En JavaScript/TypeScript se puede usar `fetch`, `axios` u otras librerías.
- En Flutter se puede usar el paquete `http`, `dio` u opciones similares.
- En Android nativo son frecuentes Retrofit, Ktor Client u OkHttp.
- En iOS se utiliza `NSURLSession` o capas superiores.

Más allá de la herramienta, el proceso general es:

Flujo de consumo de una API

Una solicitud remota tiene ida, vuelta y manejo de fallas

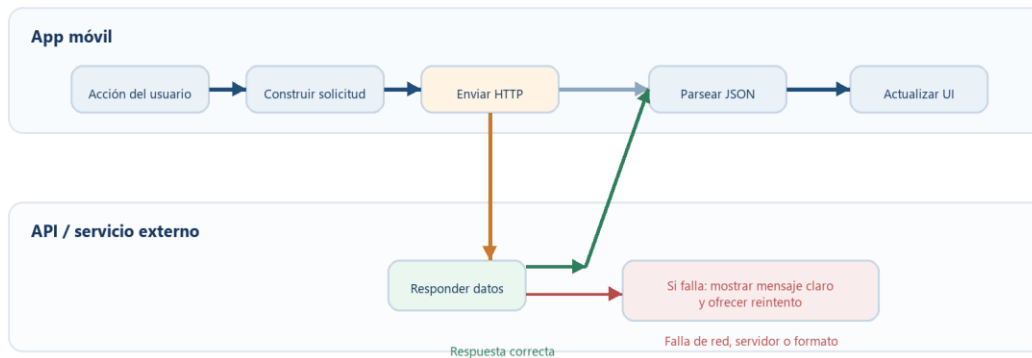


Figura 7. Secuencia básica para consumir una API desde una app móvil.

```

construir URL
-> enviar solicitud
-> esperar respuesta
-> verificar código de estado
-> parsear JSON
-> transformar datos
-> actualizar estado
  
```


6.4 Códigos de estado HTTP

Los códigos de estado ayudan a interpretar la respuesta del servidor.

Código	Significado general	Respuesta en la app
200	OK	Mostrar datos
201	Creado	Confirmar creación
400	Solicitud incorrecta	Revisar datos enviados
401	No autorizado	Pedir iniciar sesión
403	Prohibido	Informar falta de permisos
404	No encontrado	Mostrar estado vacío o recurso inexistente
500	Error del servidor	Mostrar error y permitir reintento

Aunque muchas APIs públicas devuelven siempre 200 con mensajes internos, es una buena práctica revisar el código HTTP antes de asumir que todo salió bien.

6.5 Latencia, carga y experiencia de espera

La latencia es el tiempo que transcurre entre una solicitud y su respuesta. En móvil puede variar mucho según conexión, señal, servidor y dispositivo.

La app debe diseñar la espera:

- Para cargas cortas, un indicador simple puede ser suficiente.
- Para listas o contenido visual, los skeleton loaders pueden hacer más clara la espera.
- Si ya hay datos previos, conviene mantenerlos visibles mientras se refrescan.
- Si la demora es larga, el usuario necesita entender que la app no se congeló.

Una espera sin feedback se percibe como falla.

6.6 Manejo de errores

No todos los errores son iguales. Diferenciarlos mejora la experiencia y facilita depuración.

Tipo de error	Ejemplo	Mensaje orientado al usuario
Sin conexión	El dispositivo no tiene Internet	"No hay conexión. Revisá tu red e intentá nuevamente."
Timeout	El servidor tarda demasiado	"La consulta está demorando más de lo esperado."
Recurso no encontrado	ID inexistente	"No encontramos esa información."

Tipo de error	Ejemplo	Mensaje orientado al usuario
Respuesta inválida	JSON incompleto	"Recibimos datos inesperados. Intentá más tarde."
Error del servidor	Código 500	"El servicio no está disponible en este momento."

Los mensajes técnicos como `Network request failed`, `Exception` o `null reference` no deberían llegar al usuario final. Pueden registrarse para depuración, pero la interfaz debe comunicar en lenguaje humano.

6.7 APIs públicas para practicar

Para esta unidad conviene elegir APIs que no requieran autenticación compleja y que devuelvan datos simples. Algunas categorías útiles:

- Clima por ciudad o coordenadas.
- Países, provincias o ciudades.
- Recetas, comidas o ingredientes.
- Libros, películas o personajes ficticios.
- Noticias de ejemplo o publicaciones públicas.
- Conversores de moneda o información económica de prueba.
- Datos abiertos gubernamentales o universitarios.

Al elegir una API, revisar:

- Si requiere clave.
- Si permite uso educativo gratuito.
- Si tiene documentación clara.
- Si responde en formato JSON.
- Si permite búsquedas o filtros.
- Si tiene límites de uso.
- Si los datos son apropiados para una demo académica.

6.8 Seguridad mínima al consumir servicios

Aunque la unidad se centre en APIs públicas, hay criterios básicos que deben respetarse:

- No subir claves privadas al repositorio.
- No guardar tokens sensibles en texto plano.
- No exponer datos personales reales en demos.
- Validar datos antes de mostrarlos.
- Usar HTTPS siempre que sea posible.
- No confiar ciegamente en datos recibidos desde la red.

La seguridad no empieza cuando la app se publica; empieza cuando se diseña cómo circulan los datos.

7. Estados de interfaz que no deben faltar

Una aplicación madura no diseña solo la pantalla ideal. Diseña el ciclo completo de cada experiencia.

7.1 Estado de carga

El estado de carga comunica que la app está trabajando. Puede mostrarse con:

- Spinner o indicador circular.
- Barra de progreso.
- Skeleton loader.
- Texto breve de espera.
- Deshabilitación temporal de acciones.

Debe evitarse que el usuario dude si la app recibió su acción. También hay que evitar bloquear toda la pantalla si solo se actualiza una parte.

7.2 Estado vacío

Un estado vacío ocurre cuando la operación fue correcta pero no hay datos para mostrar.

Ejemplos:

- No hay resultados para la búsqueda.
- Todavía no hay favoritos.
- No hay turnos disponibles.
- La lista se limpió por filtros demasiado estrictos.

Un buen estado vacío:

- Explica qué ocurrió.
- No culpa al usuario.
- Ofrece una acción útil.
- Mantiene el tono de la app.

Ejemplo:

No encontramos resultados para "abc".
Probá con otra búsqueda o quitá algunos filtros.

7.3 Estado de error

El estado de error debe ayudar a recuperarse. Un mensaje que solo dice "Error" no sirve.

Elementos recomendados:

- Descripción breve.
- Posible causa en lenguaje simple.
- Acción de reintento.
- Alternativa si corresponde.

Ejemplo:

No pudimos cargar los datos.
Revisá tu conexión o intentá nuevamente.
[Reintentar]

7.4 Estado de éxito

El estado de éxito no siempre necesita una pantalla especial. A veces basta con mostrar los datos actualizados. En acciones importantes conviene confirmar:

- Reserva realizada.
- Formulario enviado.
- Favorito guardado.
- Preferencia actualizada.

La confirmación debe ser proporcional a la importancia de la acción. No toda acción menor necesita un modal.

7.5 Estado de permiso denegado

Muchas apps móviles usan permisos: ubicación, cámara, notificaciones, archivos. Si el usuario los deniega, la app debe tener una alternativa.

Buenas prácticas:

- Pedir permisos solo cuando sean necesarios.
- Explicar antes para qué se usarán.
- No bloquear toda la app si el permiso no es esencial.
- Ofrecer camino manual cuando sea posible.
- Indicar cómo habilitar el permiso desde configuración si es imprescindible.

8. Organización del proyecto y versionado

El entregable de la unidad incluye código fuente versionado en GitHub. Esto no es un requisito administrativo menor: versionar permite mostrar evolución, colaborar, recuperar cambios y comunicar profesionalismo.

8.1 Estructura de carpetas

Cada framework tiene convenciones propias, pero una estructura básica debería separar responsabilidades.

Ejemplo conceptual:

```
src/
  components/
  screens/
  navigation/
  services/
  models/
  utils/
  assets/
```

Carpeta	Contenido
components	Elementos reutilizables de UI
screens	Pantallas principales de navegación
navigation	Configuración de rutas

Carpeta	Contenido
<code>services</code>	Acceso a APIs y servicios externos
<code>models</code>	Tipos, interfaces o estructuras de datos
<code>utils</code>	Funciones auxiliares
<code>assets</code>	Imágenes, íconos, fuentes u otros recursos

No es obligatorio usar exactamente estos nombres. Lo importante es que el proyecto sea legible.

8.2 README del proyecto

El repositorio debe incluir un README.md claro. Como mínimo:

- Nombre de la aplicación.
- Problema que resuelve.
- Funcionalidades implementadas.
- Tecnologías utilizadas.
- API pública consumida.
- Instrucciones para ejecutar.
- Capturas o enlace a demo si corresponde.
- Integrantes del equipo.
- Estado actual del proyecto y limitaciones conocidas.

Un buen README permite que otra persona entienda y ejecute el proyecto sin depender de explicaciones orales.

8.3 Commits

Los commits deberían representar avances comprensibles:

- Crear estructura inicial del proyecto
- Agregar navegación entre listado y detalle
- Implementar consumo de API de clima
- Agregar estados de carga y error
- Mejorar diseño de tarjetas de resultados
- Documentar ejecución en README

Evitar commits genéricos como `cosas`, `arreglos`, `final final`, `version nueva`. En un trabajo académico, el historial de commits también comunica proceso.

8.4 Ramas y colaboración

En equipos pequeños puede alcanzarse con una rama principal y coordinación clara. Si se trabaja con más integrantes, conviene separar:

- Rama para navegación.
- Rama para consumo de API.
- Rama para diseño visual.
- Rama para documentación.

Antes de unir cambios, cada integrante debe ejecutar la app y revisar que no rompió el flujo principal.

9. Pruebas y validación de aplicaciones móviles

Probar una app no es solamente verificar que "abre". La validación debe cubrir comportamiento, diseño, datos y experiencia.

Validación antes de la demo

Revisar comportamiento, datos y presentación antes de grabar

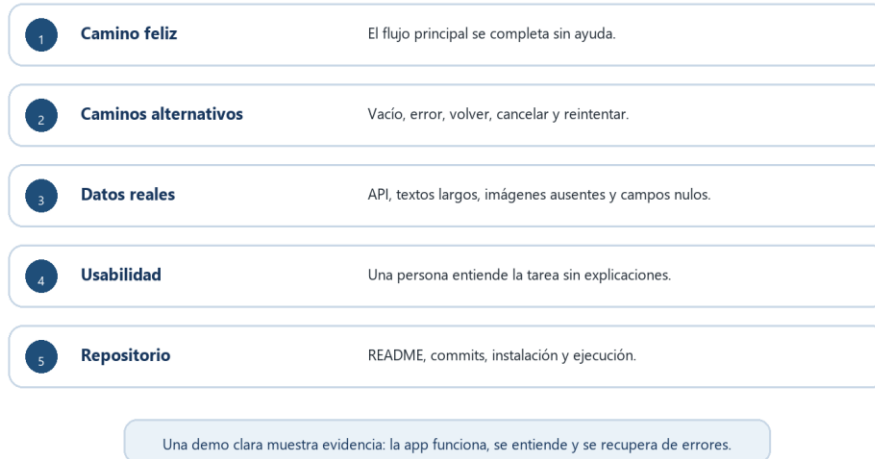


Figura 8. Checklist visual de validación antes de grabar la demo funcional.

En esta unidad se espera una validación básica pero intencional. El objetivo es detectar problemas antes de la demo.

9.1 Tipos de prueba

Tipo de prueba	Qué verifica	Ejemplo
Funcional	La app cumple una tarea	Buscar ciudad y ver detalle
Navegación	Las rutas funcionan	Volver desde detalle sin perder contexto
Estado	La UI representa cada situación	Cargando, vacío, error, éxito
Datos	La API se interpreta correctamente	Campos nulos no rompen pantalla
Usabilidad	El usuario entiende qué hacer	Puede completar tarea sin ayuda
Accesibilidad	La interfaz es usable para más personas	Contraste, tamaño táctil, etiquetas
Rendimiento básico	La app responde con fluidez	Listas no se traban, carga razonable

9.2 Prueba del camino feliz

El camino feliz es el flujo ideal en el que todo sale bien:

Abrir app -> buscar elemento -> ver resultados -> seleccionar uno -> ver detalle -> volver

Debe probarse, pero no alcanza. Muchas aplicaciones fallan porque solo se revisó este camino.

9.3 Pruebas de caminos alternativos

También hay que probar:

- Búsqueda sin resultados.
- API caída o sin conexión.
- Texto muy largo.
- Toque repetido sobre el mismo botón.
- Volver atrás desde cada pantalla.
- Rotación de pantalla si la app la permite.
- Teclado abierto sobre formularios.
- Datos incompletos.
- Primer uso de la app.

Estas situaciones revelan la calidad real de la implementación.

9.4 Checklist de validación funcional

Antes de entregar, revisar:

- La app inicia sin errores.
- La navegación principal funciona.
- Cada pantalla tiene título o contexto claro.
- La API responde y los datos se muestran correctamente.
- Existe estado de carga.
- Existe manejo de error.
- Existe estado vacío cuando corresponde.
- Los botones no generan acciones duplicadas.
- El diseño se mantiene en distintos tamaños de pantalla.
- El teclado no tapa campos importantes.
- El usuario puede volver o cancelar.
- No hay datos sensibles en el repositorio.
- El README explica cómo ejecutar el proyecto.
- La demo muestra el flujo principal sin depender de improvisación.

9.5 Validación con usuarios o pares

Una prueba rápida con compañeros puede detectar problemas que el equipo ya no ve. No hace falta un laboratorio formal. Basta con dar una tarea concreta y observar.

Guion simple:

1. Explicar que se evalúa la app, no a la persona.
2. Pedir que realice una tarea específica.
3. Observar sin guiar.

4. Registrar dónde duda, dónde se equivoca y qué esperaba encontrar.
5. Preguntar al final qué fue claro y qué resultó confuso.
6. Priorizar una o dos mejoras antes de la entrega.

Incluso una prueba con tres personas puede mostrar patrones valiosos.

10. Demo funcional

La demo de hasta 5 minutos debe mostrar que la aplicación funciona y que las decisiones tomadas tienen sentido. No conviene improvisarla. Una buena demo tiene guion.

10.1 Estructura sugerida

Tiempo aproximado	Contenido
30 segundos	Presentación del problema y objetivo de la app
60 segundos	Recorrido por navegación principal
90 segundos	Consumo de API y visualización de datos
60 segundos	Estados de carga, error o vacío
60 segundos	Decisiones de diseño y mejoras respecto del prototipo
30 segundos	Cierre: limitaciones y próximos pasos

No es necesario mostrar todo el código en la demo, salvo que se pida específicamente. Sí conviene mencionar brevemente la tecnología usada y la API integrada.

10.2 Qué debe verse en la demo

- La app iniciando correctamente.
- El flujo principal completo.
- Interacción real del usuario.
- Datos obtenidos desde una API.
- Navegación entre pantallas.
- Al menos un estado no ideal: carga, error o vacío.
- Coherencia visual con el prototipo.
- Explicación clara de qué se logró.

10.3 Errores frecuentes en demos

- Empezar explicando demasiado y usar poco la app.
- No mostrar la API en funcionamiento.
- Grabar una pantalla ilegible.
- Depender de datos cargados previamente sin explicar.
- Ocultar errores que deberían estar resueltos.
- No mostrar el repositorio o no tener README.
- Exceder el tiempo y dejar fuera lo importante.

Una demo breve y clara vale más que una presentación larga donde la aplicación casi no se ve.